Módulo II - Lógica de Programação

Prof. Roosevelt Carvalho Franklin Santos

Aula 06 | Funções e Tratamento de Erros

Objetivos da aula de hoje

Laços de repeticao

- O que são Funções (Assinatura e Escopo)
- Parâmetros e Argumentos
- Return e Composição
- O bloco Try / Catch / Finally
- Disparando erros (throw)

Problema Gerador

Contextualizando a necessidade de organização.

Cenário A (Reaproveitamento):

Na aula passada criamos um botão que rola dados. Mas agora, o chefe pediu uma nova funcionalidade: 'Quando o monstro atacar, ele também rola dados'. O que faremos? Copiar e colar aquele bloco de `for` e `Math.random` inteiro de novo? E se a regra do dado mudar? Vamos ter que mudar em 10 lugares diferentes?

Cenário B (O sistema explode):

O que acontece no nosso sistema de média se o usuário digitar 'Dois' ao invés do número 2? Ou se no jogo de dados ele pedir -5 dados? Atualmente, o código gera um `NaN` (Not a Number) e o cálculo todo quebra. O usuário vê uma tela branca ou um resultado bizarro. Como capturar esse erro antes que ele quebre o site e avisar o usuário elegantemente?

A Solução:

1. `Funções`: Encapsular lógica para ser reutilizada (chamar `rolarDado()` em qualquer lugar).
2. `Try/Catch`: Tentar executar. Se der erro, capturar e tratar sem travar o programa.

ada

Anatomia de uma função

O mini-programa

Uma função é um bloco de código nomeado que realiza uma tarefa específica.

- Assinatura: É o nome da função + os parâmetros que ela aceita.
- Parâmetros: São as variáveis "vazias" que a função espera receber para trabalhar.
- Argumentos: São os valores reais que enviamos quando chamamos a função.

Return e composição

Entrada → Processamento → Saída

Uma função pode (e deve) devolver um resultado.

O comando return encerra a função e "cospe" o valor para fora.

Composição: Usar o resultado de uma função como entrada de outra.

Exemplo de composição:

```
function calcularMedia(a, b) {
      return (a + b) / 2;
}

function mostrarNaTela(mensagem) {
 console.log(mensagem);
}

mostrarNaTela("A média é: " + calcularMedia(10, 20));
```

ada

Tratamento de Erros (Safety Net)

Prevenindo o caos

Erros acontecem (bugs, internet cai, usuário digita errado).

Não podemos deixar o erro parar o script ("crash").

- `try { ... }`: "Tente executar este bloco perigoso".
- `catch (erro) { ... }`: "Se der ruim no `try`, caia aqui e me deixe arrumar".
- `finally { ... }`: "Dando certo ou errado, execute isso no final" (Ex: esconder loading).

Criando Erros (throw)

Regras de negócio

Às vezes o dado é válido para o JS (ex: dividir por zero resulta em Infinity), mas inválido para o nosso negócio.

Podemos disparar nossos próprios erros.

`throw new Error("Mensagem explicativa").`

Isso força o código a pular imediatamente para o bloco catch.

Exemplo:

```
function dividir(a, b) {
    if (b === 0) {
        throw new Error("Não é possível dividir por zero!" );
    }
    return a / b;
}
```

ada

Bora fazer uma atividade?

"Refatorando o Jogo de Dados"

Objetivo: Transformar o código imperativo da aula passada em funções funcionais e reutilizáveis.

Além disso vamos adicionar validação na função de dados. Se o usuário pedir "0" dados ou número negativo, o sistema deve avisar o erro ao invés de mostrar resultado vazio.

Passo a Passo (Code-along):

1. Criar arquivo aula06.html.
2. Fazer a refatoração necessária.

Atividade prática

Agora é com vocês!

Crie um arquivo HTML com script para resolver os desafios abaixo. Use funções para tudo e try/catch no último exercício.

1. A Conversora (Funções Simples):
   a. Crie uma função `converterParaMaiuscula(texto)` que retorna o texto em caixa alta.
   b. Crie uma função `extrairPrimeiraLetra(texto)` que retorna só a primeira letra.
   c. Peça um nome ao usuário e mostre um alert usando composição: `alert( extrairPrimeiraLetra( converterParaMaiuscula(nome) ) )`.

2. Calculadora de Divisão (Tratamento de Erro):
   a. Crie uma função `dividir(a, b)`.
   b. Se b for igual a 0, dispare um erro: `throw new Error("Divisão impossível por zero")`.
   c. Crie uma interface (inputs e botão) que chame essa função dentro de um bloco `try/catch`.
   d. Se der erro, mostre o texto do erro em vermelho na tela. Se der certo, mostre o resultado em verde.

ada

Obrigado